

# Projet SDP – Apprentissage de modèles MR-Sort

Côme-Alexis Puech

Hugo Vigna

Salomé Fonvielle

Octobre 2026

## Préliminaires : admission d'étudiants

### Question i

Somme des poids des cours validés ( $\geq 12$ ), avec  $w = (0.25, 0.15, 0.2, 0.4)$ ,  $\lambda = 0.66$  : Alice  $0.15 + 0.2 + 0.4 = 0.75$  ( $\mathcal{A}$ ), Bob  $0.25 + 0.15 + 0.2 = 0.60$  ( $\mathcal{R}$ ), Charlie  $0.15 + 0.4 = 0.55$  ( $\mathcal{R}$ ), David  $1$  ( $\mathcal{A}$ ). Bob ne peut pas améliorer sa situation : avec 12 en physique, ce cours est déjà validé, une meilleure note n'y change rien.

### Question ii

Les 4 cours doivent jouer des rôles symétriques (3 cours quelconques suffisent), d'où  $w_M = w_P = w_L = w_H = \frac{1}{4}$  et  $\lambda \in ]\frac{1}{2}, \frac{3}{4}]$ , par exemple  $\lambda = 0.75$  : 3 cours pèsent  $\frac{3}{4} \geq \lambda$ , 2 cours pèsent  $\frac{1}{2} < \lambda$ .

### Question iii

Coalitions gagnantes (somme des poids  $\geq \lambda$ ) dans les deux cas :

| Coalition             | $w = (0.49, 0.49, 0.02)$ , $\lambda = 0.5$ | $w = (\frac{1}{3}, \frac{1}{3}, \frac{1}{3})$ , $\lambda = 0.5$ |
|-----------------------|--------------------------------------------|-----------------------------------------------------------------|
| $\{1\}, \{2\}, \{3\}$ | 0.49, 0.49, 0.02 : perdantes               | 0.33 : perdantes                                                |
| $\{1, 2\}$            | 0.98 : gagnante                            | 0.67 : gagnante                                                 |
| $\{1, 3\}, \{2, 3\}$  | 0.51 : gagnantes                           | 0.67 : gagnantes                                                |
| $\{1, 2, 3\}$         | 1 : gagnante                               | 1 : gagnante                                                    |

Dans les deux cas, un étudiant est admis si et seulement s'il valide *au moins 2 cours sur 3* : les deux jeux de paramètres définissent la même règle.

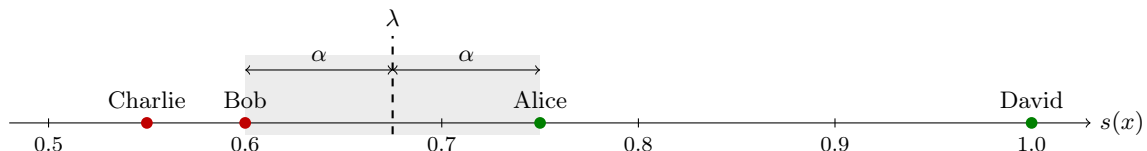
**Conclusion.**  $(w, \lambda)$  n'est pas unique : seul compte l'ensemble des coalitions gagnantes. Les poids ne mesurent pas l'importance (le cours 3, de poids 0.02, est aussi décisif que les autres) ; un modèle appris s'évalue donc sur ses décisions, pas sur ses paramètres.

### Question iv

Comme  $b$  est connu, on sait pour chaque objet  $x$  quels critères il valide :  $C(x) = \{i \in \mathcal{N} : x_i \geq b_i\}$ . Son score  $s(x) = \sum_{i \in C(x)} w_i$  est donc linéaire en  $w$ . **Variables** :  $w_i \geq 0$ ,  $\lambda \in [0, 1]$  et  $\alpha$ , la marge.

$$\begin{aligned} \max \quad & \alpha \\ \text{s.c.} \quad & s(x) \geq \lambda + \alpha & \forall x \in A^* \\ & s(x) \leq \lambda - \alpha & \forall x \in R^* \\ & \sum_{i \in \mathcal{N}} w_i = 1 \end{aligned}$$

$\alpha$  est l'écart minimal entre les scores et le seuil  $\lambda$  : on cherche le modèle qui sépare le plus nettement acceptés et rejetés. Si  $\alpha^* > 0$ , tous les objets sont bien classés ; sinon, aucun modèle ne restitue parfaitement les données.



Exemple de la question i : rejetés en rouge, acceptés en vert.

## Question v

On veut  $\delta_i(x) = \mathbb{1}_{\{x_i \geq b_i\}}$ . Si  $b$  était connu, ce serait une donnée ; mais  $b_i$  est une variable, et cette indicatrice est une fonction en escalier de  $b_i$ , qu'on ne peut pas écrire telle quelle dans un programme linéaire. On l'impose par deux contraintes linéaires, une pour chaque sens de l'équivalence ( $\forall i \in \mathcal{N}, \forall x$ ) :

$$\begin{aligned} x_i - b_i &\geq M(\delta_i(x) - 1) & (1) \quad \delta_i(x) = 1 \Rightarrow x_i \geq b_i \\ x_i - b_i &\leq M\delta_i(x) - \varepsilon & (2) \quad \delta_i(x) = 0 \Rightarrow x_i < b_i \end{aligned}$$

On suppose les données normalisées dans  $[0, 1]$  sur chaque critère (transformation croissante, qui ne change pas les affectations). Alors  $b_i \in [0, 1]$ ,  $|x_i - b_i| \leq 1$ , et  $M = 2$  suffit pour que la contrainte qui ne doit pas jouer soit toujours vraie.  $\varepsilon > 0$  petit remplace l'inégalité stricte.

|                   | Contrainte (1)                       | Contrainte (2)                                    | Conclusion     |
|-------------------|--------------------------------------|---------------------------------------------------|----------------|
| $\delta_i(x) = 1$ | $x_i - b_i \geq 0$                   | $x_i - b_i \leq M - \varepsilon$ (toujours vraie) | $x_i \geq b_i$ |
| $\delta_i(x) = 0$ | $x_i - b_i \geq -M$ (toujours vraie) | $x_i - b_i \leq -\varepsilon$                     | $x_i < b_i$    |

## Question vi

On veut  $w_i(x) = w_i \delta_i(x)$  : c'est un produit de deux variables, donc non linéaire. Les contraintes

$$w_i \geq w_i(x) \geq 0 \quad (3)$$

$$\delta_i(x) \geq w_i(x) \geq \delta_i(x) + w_i - 1 \quad (4)$$

remplacent ce produit. On vérifie les deux cas ; les bornes non citées sont toujours vraies car  $0 \leq w_i \leq 1$  ( $w_i \geq 0$  et  $\sum_i w_i = 1$ ) :

|                   | Borne inférieure      | Borne supérieure      | Conclusion     |
|-------------------|-----------------------|-----------------------|----------------|
| $\delta_i(x) = 1$ | $w_i(x) \geq w_i$ (4) | $w_i(x) \leq w_i$ (3) | $w_i(x) = w_i$ |
| $\delta_i(x) = 0$ | $w_i(x) \geq 0$ (3)   | $w_i(x) \leq 0$ (4)   | $w_i(x) = 0$   |

Donc  $\sum_{i \in \mathcal{N}} w_i(x) = \sum_{i: x_i \geq b_i} w_i = s(x)$  : c'est le poids de la coalition des critères validés par  $x$ , c'est-à-dire son score.

L'étudiant  $x$  est admis si et seulement si

$$s(x) = \sum_{i \in \mathcal{N}} w_i(x) \geq \lambda.$$

## Question vii

Les questions v et vi permettent d'écrire le score sans connaître  $b$  :

$$s(x) = \sum_{i \in \mathcal{N}} w_i \delta_i(x) = \sum_{i \in \mathcal{N}} w_i(x).$$

On reprend donc le PL de la question iv avec ce score, en ajoutant les contraintes (1) à (4) qui le définissent. Les données sont normalisées dans  $[0, 1]$ ,  $M = 2$  et  $\varepsilon > 0$  petit.

### Variables.

- du modèle :  $w_i \geq 0$ ,  $b_i \in [0, 1]$  pour  $i \in \mathcal{N}$ , et  $\lambda \in [0, 1]$  ;
- de la marge :  $\alpha$  ;
- pour chaque objet  $x$  et chaque critère  $i$  :  $\delta_i(x) \in \{0, 1\}$  et  $w_i(x) \geq 0$ .

### Programme.

$$\begin{aligned} \max \quad & \alpha \\ \text{s.c.} \quad & \sum_{i \in \mathcal{N}} w_i(x) \geq \lambda + \alpha & \forall x \in A^* & \text{(acceptés)} \\ & \sum_{i \in \mathcal{N}} w_i(x) \leq \lambda - \alpha & \forall x \in R^* & \text{(rejetés)} \\ & x_i - b_i \geq M(\delta_i(x) - 1) & \forall i, \forall x & (1) \\ & x_i - b_i \leq M\delta_i(x) - \varepsilon & \forall i, \forall x & (2) \\ & w_i \geq w_i(x) \geq 0 & \forall i, \forall x & (3) \\ & \delta_i(x) \geq w_i(x) \geq \delta_i(x) + w_i - 1 & \forall i, \forall x & (4) \\ & \sum_{i \in \mathcal{N}} w_i = 1 \end{aligned}$$

Les variables  $\delta_i(x)$  étant binaires, c'est un programme linéaire mixte en nombres entiers (MILP), avec  $n|A^* \cup R^*|$  variables binaires. Comme à la question iv,  $\alpha^* > 0$  signifie que le modèle appris restitue tout le learning set.

## Question viii – Résolution avec Gurobi

Voir la partie *Jalon 1* ci-dessous : implémentation, exemple et protocole de test.

## Question viii bis – Généralisation à $p$ catégories

Les catégories sont ordonnées, de  $\mathcal{C}^1$  (la pire) à  $\mathcal{C}^p$  (la meilleure). On apprend  $p - 1$  frontières  $b^1, \dots, b^{p-1}$  :  $b^h$  sépare  $\mathcal{C}^h$  de  $\mathcal{C}^{h+1}$ . Les poids  $w$  et le seuil  $\lambda$  sont communs à toutes les frontières. Un objet  $x$  est dans  $\mathcal{C}^h$  s'il franchit la frontière inférieure  $b^{h-1}$  mais pas la frontière supérieure  $b^h$  : on applique à chaque frontière le raisonnement à deux classes.

On définit donc  $\delta_i^h(x)$  et  $w_i^h(x)$  pour chaque frontière  $h$ , avec les contraintes (1) à (4) où  $b$  est remplacé par  $b^h$ . Le score de  $x$  par rapport à  $b^h$  est  $s_h(x) = \sum_{i \in \mathcal{N}} w_i^h(x)$ .

**Programme.**

$$\begin{aligned}
\max \quad & \alpha \\
\text{s.c.} \quad & s_{h-1}(x) = \sum_{i \in \mathcal{N}} w_i^{h-1}(x) \geq \lambda + \alpha & \forall x \in \mathcal{C}^h, h \geq 2 & \text{(franchit } b^{h-1}) \\
& s_h(x) = \sum_{i \in \mathcal{N}} w_i^h(x) \leq \lambda - \alpha & \forall x \in \mathcal{C}^h, h \leq p - 1 & \text{(ne franchit pas } b^h) \\
& (1) \text{ à } (4) \text{ avec } b^h, \delta_i^h(x), w_i^h(x) & \forall i, \forall x, \forall h \\
& b_i^{h-1} \leq b_i^h & \forall i, 2 \leq h \leq p - 1 & \text{(frontières ordonnées)} \\
& \sum_{i \in \mathcal{N}} w_i = 1
\end{aligned}$$

Pour  $p = 2$ , on retrouve le programme de la question vii. En pratique, seules les frontières  $b^{h-1}$  et  $b^h$  qui encadrent la catégorie de  $x$  interviennent dans ses contraintes : on ne crée  $\delta_i^h(x)$  et  $w_i^h(x)$  que pour ces deux frontières, soit au plus  $2n$  variables binaires par objet. C'est ce que fait `learn_mrsort` (résultats pour  $p = 3$  dans la partie Jalon 1).

## Jalon 1 : implémentation et validation

### Le programme

Le MILP de la question vii, généralisé à  $p$  catégories (question viii bis), est implémenté en Python avec `gurobipy` dans le fichier `src/mrsort.py` (code et mode d'emploi sur le dépôt du projet).

- **Entrée** : un learning set, fichier CSV avec une ligne par objet : les  $n$  performances, puis la classe. Le nombre de critères  $n$  et le nombre de catégories  $p$  sont déduits du fichier (ou  $p$  est donné).
- **Traitement** : normalisation des données dans  $[0, 1]$ , construction du MILP, résolution par Gurobi.
- **Sortie** : les poids  $w$ , le seuil  $\lambda$ , les frontières  $b^1, \dots, b^{p-1}$  (dans l'échelle d'origine), la marge  $\alpha^*$  et le taux de restitution du learning set.

Commande : `python src/mrsort.py learn learning_set.csv`.

### Exemple

Learning set de 60 étudiants notés sur 20 dans 4 cours, répartis en 3 catégories par un modèle de référence. Le programme trouve en moins d'une seconde un modèle qui restitue les 60 affectations ( $\alpha^* = 0.1$ ). Ses frontières sont proches de celles du modèle de référence sans être identiques (par exemple  $b^1 = (8.1, 7.5, 15.4, 2.5)$  contre  $(9.0, 7.4, 15.7, 2.6)$ ) : comme vu à la question iii, plusieurs modèles expliquent les mêmes données.

### Modèle appris sur Dataset0

Dataset0 : 60 objets, 5 critères à maximiser, 3 catégories. Le programme trouve en moins d'une seconde un modèle qui restitue les 60 affectations ( $\alpha^* = 0.071$ ) :

|                                   | crit1 | crit2 | crit3 | crit4 | crit5 |                                 |
|-----------------------------------|-------|-------|-------|-------|-------|---------------------------------|
| Poids $w_i$                       | 1/7   | 2/7   | 1/7   | 1/7   | 2/7   | $\lambda = 0.643 \approx 4.5/7$ |
| Frontière $b^1$ (catégorie 1 / 2) | 34.5  | 35    | 33    | 34.5  | 33    |                                 |
| Frontière $b^2$ (catégorie 2 / 3) | 41.5  | 49    | 33    | 43.5  | 49    |                                 |

**Lecture.** Les critères 2 et 5 comptent double. Un objet franchit une frontière s’il l’atteint sur des critères totalisant au moins 5 « points » sur 7 (par exemple crit2, crit5 et un autre critère). Il est en catégorie 3 s’il franchit  $b^2$ , en catégorie 2 s’il franchit  $b^1$  seulement, en catégorie 1 sinon. Comme vu à la question iii, ce modèle n’est pas le seul compatible avec les données : d’autres paramètres donneraient les mêmes 60 affectations. Sur chaque critère, toute position de la frontière entre deux valeurs voisines des données donne les mêmes affectations ; Gurobi la place au hasard dans cet intervalle. On la recentre au milieu de l’intervalle, ce qui ne change aucune affectation et l’éloigne au maximum des données, pour mieux classer de nouveaux objets. Commande : `python src/mrsort.py learn data/dataset0.csv`.

## Protocole de test

Pour mesurer la qualité de l’apprentissage, on part d’un modèle connu et on vérifie qu’on le retrouve :

1. On tire au hasard un modèle MR-Sort de référence  $\mathcal{M}'$  : poids,  $\lambda \in [0.5, 1]$ , frontières ordonnées.
2. On tire  $|L|$  objets aléatoires dans  $[0, 1]^4$  et on les classe avec  $\mathcal{M}'$  : c’est le learning set  $L$ .
3. On apprend  $\mathcal{M}^*$  sur  $L$  avec notre programme.
4. On tire 1000 nouveaux objets  $T$ , jamais vus, et on les classe avec  $\mathcal{M}'$  et  $\mathcal{M}^*$ . La *qualité* est la proportion d’objets affectés à la même classe.

On répète 20 fois et on fait la moyenne. On teste sur des objets nouveaux car, sur  $L$ , la restitution est parfaite par construction : la vraie question est de savoir si  $\mathcal{M}^*$  décide comme  $\mathcal{M}'$  sur des cas inconnus. Commande : `python src/mrsort.py test -n 4 -p 2 -m 50 --runs 20`.

## Résultats

| $p$ | $ L $ | Qualité moyenne sur $T$ | Écart-type | Temps moyen |
|-----|-------|-------------------------|------------|-------------|
| 2   | 20    | 88.5 %                  | 5.4 %      | 0.05 s      |
| 2   | 50    | 96.4 %                  | 2.5 %      | 0.18 s      |
| 2   | 100   | 98.4 %                  | 1.2 %      | 0.49 s      |
| 3   | 50    | 91.5 %                  | 4.3 %      | 0.31 s      |
| 3   | 100   | 96.6 %                  | 2.0 %      | 0.74 s      |

Dans les 100 essais,  $\alpha^* > 0$  et le learning set est restitué à 100 %. La qualité sur  $T$  augmente avec  $|L|$  : plus il y a d’exemples, plus les frontières et les coalitions gagnantes sont contraintes, et plus  $\mathcal{M}^*$  se rapproche de  $\mathcal{M}'$ . Avec 3 catégories, il faut plus d’exemples pour une qualité comparable, car il y a deux frontières à apprendre.